



南開大學

《多线程编程实验》

矩阵 SVD 分解算子 Pthread 与 OpenMP 多线程优化实验
报告

姓名：尚文轩

学号：2412456

专业：计算机科学与技术

2026 年 5 月 23 日

目录

1	GKH 迭代的可并行性分析	3
1.1	活动块与 block 级任务	3
1.2	block 内部的顺序依赖	3
1.3	数据竞争风险与范围化更新	4
1.4	无锁并行设计与同步边界	5
2	多线程优化设计与实现	5
2.1	整体设计思路	5
2.2	并行模式宏定义	6
2.3	任务列表构造	6
2.4	OpenMP block 级并行	7
2.5	OpenMP 调度策略分析	7
2.6	adaptive threshold 控制无效并行	8
2.7	Pthread dynamic 原子任务队列	8
2.8	Pthread static 固定任务分配	9
2.9	线程数控制与主线程参与计算	10
2.10	cleanup 阶段并行化	10
2.11	并行化策略小结	11
3	实验环境与测试规范	11
3.1	服务器正式测试环境	11
3.2	本地 VTune 分析环境	12
3.3	VTune 编译脚本	12
4	正确性验证与服务器性能结果	13
4.1	正确性验证	13
4.2	OpenMP naive static 与 adaptive static 对比	14
4.3	OpenMP 调度策略对比	14
4.4	Pthread dynamic 与 Pthread static 对比	15
4.5	block evolution profile 分析	15
4.6	parallel cleanup 线程扩展性	15
4.7	最终版本对比	16
5	VTune 性能剖析	16
5.1	VTune 分析目的与环境说明	16
5.2	Hotspots 分析	17
5.3	Threading 分析	18
5.4	Microarchitecture Exploration 分析	19
5.5	Memory Access 分析	20
5.6	VTune 结果与服务器实验的对应关系	22
5.7	VTune 结论	22
6	总结与思考	22
6.1	对算法并行性的认识	23
6.2	对 naive 并行的反思	23
6.3	对 OpenMP 与 Pthread 调度策略的认识	23
6.4	对 cleanup 并行化的认识	24
6.5	对 VTune 性能分析的认识	24
6.6	局限性与后续优化方向	24
7	结论	25

A	Git 项目链接	26
B	实验结果文件组织	26
C	AI 辅助编程对话记录	27
C.1	GKH 迭代是否可以直接整体并行化	27
C.2	为什么需要 range-limited B update	28
C.3	OpenMP naive static 版本为什么反而变慢	28
C.4	OpenMP static、dynamic 和 guided 应该如何比较	28
C.5	Pthread dynamic 是否比 static 更合适	28
C.6	主线程是否应该参与 Pthread 计算	28
C.7	cleanup 阶段为什么适合 OpenMP 行级并行	29
C.8	VTune 在服务器上无法使用时如何处理	29
C.9	本地 VTune 结果和服务器结果不一致时如何解释	29
C.10	如何解释 VTune 中多线程版本 CPU 利用率仍然很低	29
C.11	如何理解 <code>apply_right_cols</code> 长期占据热点	30
C.12	后续如果继续优化应该从哪里入手	30
D	参考文献	30

1 GKH 迭代的可并行性分析

1.1 活动块与 block 级任务

在上二对角矩阵 B 中, 除对角线元素外, 理论上只有超对角线元素 $B_{k,k+1}$ 可能非零。GKH 迭代的目标正是逐步使这些超对角线元素收敛到零。如果某个超对角线元素 $B_{k,k+1}$ 已经足够小, 则它可以被视为数值上的零。此时矩阵 B 在该位置被分裂为左右两个相互独立的子问题, 后续迭代可以分别在两个子块上进行。

程序中的 `split_active_blocks` 函数基于如下判据判断超对角线元素是否可以置零:

$$|B_{k,k+1}| \leq \text{tol} \cdot (|B_{k,k}| + |B_{k+1,k+1}| + 1). \quad (1)$$

若满足该条件, 则将 $B_{k,k+1}$ 置为零, 并认为该位置左右两侧已经解耦。随后程序从左到右扫描超对角线, 将连续未收敛的部分组织为若干活动块。每个活动块用闭区间 $[l, r]$ 表示, 含义是从第 l 行/列到第 r 行/列之间仍然属于同一个尚未完全收敛的上二对角子问题。

若某个活动块满足 $r = l$, 说明该块已经退化为单个对角元, 不需要继续进行 GKH 迭代。若 $r > l$, 则该块是非平凡活动块, 需要执行一次 `one_block_step`。因此, 每一轮 GKH 迭代都可以分解为如下过程:

- (1) 清理上二对角结构之外的数值噪声;
- (2) 处理对角线近零但超对角线仍非零的异常情况;
- (3) 根据超对角线零元素划分活动块;
- (4) 将所有非平凡活动块收集为任务列表;
- (5) 对任务列表中的 block 执行一次 GKH bulge chasing。

从并行程序设计角度看, 第四步得到的非平凡活动块列表就是天然的任务集合。不同活动块之间由于已经被零超对角线分隔, 数学上对应不同的二对角子问题, 因此可以作为相互独立的任务并行处理。本文的 OpenMP 和 Pthread 版本均以这一 block 级任务列表作为并行化对象。

1.2 block 内部的顺序依赖

虽然不同活动块之间可以并行, 但单个活动块内部的 `one_block_step` 不能简单地并行化。这是因为 Golub–Kahan SVD iteration 的核心过程是 bulge chasing, 即先在活动块左端引入一个 bulge, 然后通过一系列交替的右 Givens 旋转和左 Givens 旋转, 将该 bulge 从块左端逐步追赶到块右端。

对于活动块 $[l, r]$, 一次 `one_block_step` 的主要流程如下:

- (1) 根据活动块右下角 2×2 子问题计算 Wilkinson shift;
- (2) 使用 $B_{l,l}^2 - \mu$ 和 $B_{l,l}B_{l,l+1}$ 构造首次右 Givens 旋转;
- (3) 对 B 和 V 应用该右旋转, 在活动块左端引入 bulge;
- (4) 使用左 Givens 旋转消去新产生的次对角元素;
- (5) 对 $k = l + 1, \dots, r - 1$ 依次执行右旋转和左旋转, 使 bulge 沿着二对角结构向右传播;
- (6) 最终将 bulge 追赶出活动块, 使该 block 保持近似上二对角结构。

该过程具有明显的数据依赖。例如，在第 k 步构造右 Givens 旋转时，需要读取前一步左旋转之后更新得到的 $B_{k-1,k}$ 和 $B_{k-1,k+1}$ ；随后构造左 Givens 旋转时，又需要读取当前右旋转之后产生的 $B_{k,k}$ 和 $B_{k+1,k}$ 。因此，循环中后一位置的旋转参数依赖前一位置已经更新过的矩阵元素。如果强行将 `one_block_step` 内部的 k 循环并行化，不同线程会同时修改存在依赖关系的矩阵元素，从而破坏 bulge chasing 的数学过程，并可能导致错误的奇异值分解结果。

因此，本文没有选择 block 内部并行，而是选择更安全的 block 间并行。这种设计牺牲了单个大 block 内部的并行机会，但能够保持 GKH 迭代的数值正确性。后续实验也表明，当前测试样例中大多数迭代只有一个非平凡活动块，因此 block 级并行度本身非常有限，这成为多线程版本难以获得线性加速的主要原因。

1.3 数据竞争风险与范围化更新

理论上，不同活动块之间相互独立，但在实际代码实现中仍然需要谨慎处理矩阵写入范围。原始的行旋转与列旋转函数为：

- `apply_left_rows`: 对矩阵的两整行执行左 Givens 旋转；
- `apply_right_cols`: 对矩阵的两整列执行右 Givens 旋转。

如果在 block 并行阶段直接使用这两个全范围函数，那么不同 block 虽然逻辑上对应不同子问题，却可能在更新 B 时写入对方所在的行或列区域。例如，一个 block 中的右旋转只应影响该 block 内部的局部二对角结构；如果直接对整列执行旋转，就可能改写其他 block 对应行上的元素。这种写入不仅会破坏分块后的独立性，还会引入数据竞争。

为了解决这一问题，本文在 `one_block_step` 中对 B 的更新改为范围化操作：

- `apply_right_cols_range`: 只在指定行区间 $[l, r]$ 内更新两列；
- `apply_left_rows_range`: 只在指定列区间 $[l, r]$ 内更新两行。

这样，对于活动块 $[l, r]$ ，所有对 B 的更新都被限制在该 block 对应的局部子矩阵范围内。不同活动块的区间互不重叠，因此它们在并行执行时不会写入同一个 B 元素。这也是本文能够在 block 并行阶段避免全局互斥锁的重要前提。

需要注意的是，范围化更新主要用于 B 。对于 U 和 V ，程序仍然调用全矩阵更新函数来累积正交变换。这是因为维护 SVD 恒等式

$$A = UBV^T \quad (2)$$

要求每次对 B 施加左旋或右旋后，都要同步更新 U 或 V 。具体来说，若 $B \leftarrow LB$ ，为了保持分解恒等式不变，需要令

$$U \leftarrow UL^T. \quad (3)$$

若 $B \leftarrow BR$ ，则需要同步更新

$$V \leftarrow VR. \quad (4)$$

在 block 并行阶段，不同活动块对应的旋转索引区间互不重叠。因此，不同线程对 U 和 V 的更新虽然会遍历多行，但作用的列对不同。例如 block $[l_1, r_1]$ 只会更新 U, V 中与 $l_1, \dots, r_1$ 相关的列，而另一个 block $[l_2, r_2]$ 若与其不重叠，则不会写入同一列。因此，不同 block 对 U 和 V 的写入也不存在同一元素上的并发写冲突。

1.4 无锁并行设计与同步边界

基于上述分析，本文最终采用无全局锁的 block 并行设计。这里的“无锁”并不是指整个程序没有同步，而是指在 block 任务执行过程中，不对矩阵 B 、 U 、 V 的每次更新加互斥锁。线程安全主要由以下条件保证：

- (1) 活动块由零超对角线分隔，不同 block 对应互不重叠的子问题；
- (2) 对 B 的更新使用 range-limited 函数，保证只写入当前 block 的局部范围；
- (3) 对 U 和 V 的更新虽然作用于整列，但不同 block 使用互不重叠的列索引；
- (4) 每一轮 GKH 迭代结束后，必须等待所有 block 任务完成，才能进入下一轮 cleanup、zero handling 和 split。

在 OpenMP 版本中，`#pragma omp parallel for` 结束时存在隐式 barrier，保证所有 block 任务完成后才进入下一轮迭代。在 Pthread 版本中，主线程通过 `pthread_join` 等待所有 worker 线程结束。因此，本文避免的是矩阵更新过程中的细粒度锁，而不是取消迭代轮次之间必要的同步。

这种设计的优点是实现简单、锁开销低，并且不会因为频繁加锁破坏数值计算循环的性能。但它也有明显局限：可并行任务数量完全取决于当前 GKH 迭代中非平凡活动块的数量。当每轮只有一个非平凡 block 时，即使创建了多个线程，也只有一个线程能够执行有效计算，其余线程只能等待。这正是后续实验中 OpenMP 和 Pthread block 级并行加速有限的根本原因。

2 多线程优化设计与实现

2.1 整体设计思路

本文的多线程优化并不是直接对 GKH 迭代中的所有循环使用并行指令，而是先根据算法依赖关系将可并行部分和必须串行部分分离出来。GKH 迭代中，单个活动块内部的 bulge chasing 存在严格的数据依赖，因此不适合直接并行化；而不同活动块之间由已经收敛为零的超对角线元素分隔，可以视为相互独立的子问题。因此，本文的核心并行策略是：

- (1) 每轮 GKH 迭代先调用 `split_active_blocks` 生成活动块列表；
- (2) 过滤掉已经收敛的单点块，只保留 $r > l$ 的非平凡活动块；
- (3) 将非平凡活动块组织为任务数组 `tasks`；
- (4) 使用 OpenMP 或 Pthread 对 `tasks` 中的 block 进行并行处理；
- (5) 每一轮 block 处理结束后，线程同步，再进入下一轮 GKH 迭代。

这种设计属于任务级并行，而不是单纯的循环级并行。它的优点是不会破坏 `one_block_step` 内部的数值依赖关系，正确性相对容易保证；缺点是可并行任务数量完全取决于当前迭代中非平凡活动块的数量。如果某一轮只有一个非平凡 block，则无论使用多少线程，都只能有一个线程执行有效计算。

2.2 并行模式宏定义

为了在不大规模改动测试脚本的前提下比较多个实现版本,实验通过编译期宏 `SVD_PARALLEL_MODE` 控制不同并行策略。这样可以使用同一份源代码编译出串行、OpenMP 和 Pthread 多个版本,避免维护多份代码导致实现不一致。各模式定义如表 1 所示。

表 1: 并行模式定义

模式	含义
0	串行 block processing
1	OpenMP static block processing
2	OpenMP dynamic block processing, chunk size = 1
3	OpenMP guided block processing, chunk size = 1
4	Pthread dynamic block processing with atomic task index
5	Pthread static block processing with fixed task assignment

除了 `SVD_PARALLEL_MODE` 外,实验还使用以下宏控制运行行为:

- `SVD_NUM_THREADS`: 控制 OpenMP 或 Pthread 使用的线程数量;
- `SVD_MIN_PARALLEL_TASKS`: 控制当前轮次至少有多少个 block task 时才启用并行;
- `SVD_PARALLEL_CLEANUP`: 控制是否对 `cleanup_bidiagonal` 使用 OpenMP 行级并行;
- `SVD_CLEANUP_MIN_ELEMENTS`: 控制 cleanup 并行化的最小矩阵规模阈值。

这种宏控制方式便于进行消融实验。例如只改变 `SVD_PARALLEL_MODE` 可以比较不同调度策略;只改变 `SVD_NUM_THREADS` 可以进行线程扩展性实验;只改变 `SVD_PARALLEL_CLEANUP` 可以评估 cleanup 并行化的单独贡献。

2.3 任务列表构造

每轮 GKH 迭代得到活动块后,程序会将非平凡活动块提取为任务数组:

Listing 1: 非平凡活动块任务列表构造

```

1  std::vector<Block> tasks;
2  tasks.reserve(blocks.size());
3
4  for (int i = static_cast<int>(blocks.size()) - 1; i >= 0; --i)
5  {
6      if (blocks[i].r > blocks[i].l)
7      {
8          tasks.push_back(blocks[i]);
9      }
10 }
```

这里仍然沿用串行版本中从右到左处理 block 的顺序。这样做一方面保持与原始串行算法尽可能接近,便于比较正确性和性能;另一方面也减少了末端 block 对前面 block 的潜在影响。在并行版本中,任务数组中的 block 在数学上已经互相独立,因此 OpenMP 或 Pthread 可以对这些任务重新分配执行。

2.4 OpenMP block 级并行

OpenMP 是共享内存多线程编程中常用的接口，其优势在于可以通过编译指示对循环进行并行化，而不需要程序员手动管理线程创建与销毁 [1]。在本文中，每一轮 GKH 迭代得到的 `tasks` 数组天然适合使用 `omp parallel for` 处理。静态调度版本的核心结构如下：

Listing 2: OpenMP static block 级并行核心代码

```
1  #pragma omp parallel for schedule(static)
2  for (int task_id = 0; task_id < task_count; ++task_id)
3  {
4      one_block_step(U, B, V, tasks[task_id].l, tasks[task_id].r);
5  }
```

该实现的并行粒度是 block，而不是 `one_block_step` 内部的每一次 Givens 旋转。换言之，一个线程拿到一个 block 后，会完整执行该 block 内部的 bulge chasing。这样可以避免破坏 block 内部的顺序依赖，也避免在每个 Givens 旋转级别引入过细的任务划分。

2.5 OpenMP 调度策略分析

为了分析不同 OpenMP 调度策略对 GKH block 任务的影响，本文实现并比较了 `static`、`dynamic` 和 `guided` 三种调度方式。虽然 OpenMP 还支持 `runtime`、`auto` 等调度方式，但为了保持实验可控，本文重点选择这三种具有代表性的策略。

static 调度。 `schedule(static)` 会在进入并行循环时一次性将任务划分给各个线程。其优点是调度开销低，任务分配确定，适合每个循环迭代耗时接近的情况。对于 GKH block 任务，如果每个 block 大小相近，则 static 调度可能表现较好。然而实际 GKH 迭代中，不同活动块的长度 $r - l + 1$ 可能不同，`one_block_step` 的耗时也会随 block 大小变化，因此 static 调度可能出现某些线程提前结束而其他线程仍在处理大 block 的负载不均问题。

dynamic 调度。 `schedule(dynamic, 1)` 让线程每次动态领取一个任务。其优点是负载均衡能力较强，适合任务大小不均的场景。对于 GKH block 任务，dynamic 调度可以在少数多 block 迭代中让完成较小 block 的线程继续领取后续 block，从而减少负载不均。缺点是每次任务领取都需要 OpenMP runtime 参与调度，调度开销高于 static。当任务数量很少时，这种开销可能超过并行收益。

guided 调度。 `schedule(guided, 1)` 在循环开始阶段分配较大的 chunk，后期逐渐减小 chunk size，试图在调度开销和负载均衡之间取得折中。对于任务数量较多且任务耗时不完全均匀的循环，guided 调度通常比 static 更灵活，又比 dynamic 具有更低的调度开销。但在本文的 GKH block 并行中，很多轮次的 task 数量只有 1，因此 guided 调度也无法从根本上解决并行任务不足的问题。

三种策略的理论特点总结如表 2 所示。

表 2: OpenMP 调度策略设计分析

调度方式	优点	缺点	对 GKH block 任务的预期
static	开销低, 分配确定	负载不均时效果差	适合作为低开销基准
dynamic(1)	负载均衡好	任务领取开销较高	少数多 block 迭代可能更优
guided(1)	兼顾开销和均衡	task 很少时优势有限	可能略优于 static/dynamic

后续实验结果表明, dynamic 和 guided 相比 static 有轻微优势, 但三者差距很小。这与上述分析一致: 调度策略只能优化已有任务的分配方式, 而不能增加每轮 GKH 迭代中可并行 block 的数量。

2.6 adaptive threshold 控制无效并行

初始 naive OpenMP static 版本在每轮 GKH 迭代中都进入 OpenMP 并行区。然而 profiling 后发现, 绝大多数迭代中只有一个非平凡活动块。在这种情况下, 即使创建多个线程, 也只有一个任务可以执行, 其他线程只能等待。并行区创建、任务调度和线程同步的固定开销反而会使程序变慢。

为此, 本文引入自适应阈值 `SVD_MIN_PARALLEL_TASKS`。只有当当前轮次的非平凡 block 数量达到阈值时, 才启用 OpenMP 或 Pthread block 并行; 否则直接串行执行任务列表。核心逻辑如下:

Listing 3: adaptive block 并行判断

```

1  const int task_count = static_cast<int>(tasks.size());
2  const bool use_parallel_blocks = task_count >= SVD_MIN_PARALLEL_TASKS;
3
4  if (use_parallel_blocks)
5  {
6      // parallel block scheduling
7  }
8  else
9  {
10     // serial block processing
11 }

```

本实验中 `SVD_MIN_PARALLEL_TASKS` 取 2。其含义是: 只有当当前迭代至少存在两个非平凡活动块时, 才进入并行 block 调度; 若只有一个任务, 则直接由当前线程串行执行。这个阈值选择比较保守, 但符合 GKH block profiling 结果。由于大多数迭代只有一个非平凡 block, 阈值判断可以避免大量无效并行区开销。

需要强调的是, adaptive threshold 并不会改变算法计算量, 也不会改变 GKH 的数值过程。它只是在运行时根据任务数量决定是否值得启用多线程, 因此属于调度层面的优化。

2.7 Pthread dynamic 原子任务队列

POSIX Threads 是更底层的线程编程接口, 需要程序员显式创建线程、传递参数和管理线程同步 [2, 3]。与 OpenMP 不同, Pthread 不提供自动的 `parallel for` 调度机制, 因此需要手动设计任务分配方式。

本文实现的 Pthread dynamic 版本使用一个共享原子变量 `next_task` 作为任务指针。每个 worker 线程不断通过 `fetch_add` 获取一个任务编号，直到任务全部完成。核心代码如下：

Listing 4: Pthread dynamic 原子任务领取

```

1  while (true)
2  {
3      const int task_id = ctx->next_task.fetch_add(1, std::memory_order_relaxed);
4      if (task_id >= task_count)
5      {
6          break;
7      }
8
9      const Block &blk = *(ctx->tasks)[task_id];
10     one_block_step(*(ctx->U), *(ctx->B), *(ctx->V), blk.l, blk.r);
11 }

```

该方法类似一个简单的动态任务队列。由于每次领取任务只需要递增一个原子整数，不需要维护复杂的队列结构，因此实现较简单。这里使用 `std::memory_order_relaxed`，原因是原子变量只用于给任务编号，不用于同步矩阵数据本身。任务之间的矩阵写入安全性由 block 区间不重叠和 range-limited 更新保证。

Pthread dynamic 的优点是负载均衡能力较好。如果某个 block 较大，执行时间更长，其他线程完成自己的任务后仍可继续领取后续任务。缺点是每个任务领取都需要一次原子操作，并且本实验实现采用的是“需要并行时创建线程，执行完后 join”的动态线程方式，而不是程序启动时创建一组长期存在的线程池。因此，当可并行任务数量很少时，线程创建和销毁成本也可能抵消收益。

2.8 Pthread static 固定任务分配

为了与 dynamic 任务队列对比，本文还实现了 Pthread static 调度。其任务分配规则为：

$$\text{task } i \rightarrow \text{thread } (i \bmod T), \quad (5)$$

其中 T 为线程数。代码中采用轮转分配方式：

Listing 5: Pthread static 固定任务分配

```

1  for (int task_id = worker->thread_id;
2      task_id < task_count;
3      task_id += worker->thread_count)
4  {
5      const Block &blk = *(worker->tasks)[task_id];
6      one_block_step(*(worker->U), *(worker->B), *(worker->V), blk.l, blk.r);
7  }

```

Pthread static 的优点是没有原子任务领取开销，任务分配规则也比较简单。其缺点是负载均衡能力弱。如果某些 block 明显大于其他 block，被分到大 block 的线程会执行更久，而其他线程即使已经完成任务也不能继续领取新任务。对于 GKH 迭代而言，block 大小会随着超对角线收敛动态变化，因此 static 调度理论上不如 dynamic 灵活。

本文保留 Pthread static 版本，主要是为了形成对照实验：如果 dynamic 明显优于 static，则说明负载均衡是主要影响因素；如果二者差距很小，则说明任务数量不足才是主要限制。后续实验

显示, Pthread dynamic 仅略优于 static, 因此 GKH block 级并行的主要瓶颈仍然是非平凡 block 数量过少。

2.9 线程数控制与主线程参与计算

课程服务器每个任务最多使用 8 个核心。为了避免误设过大的线程数, 同时保证 1、2、4、8 线程扩展性实验可控, 本文将线程数封装为 `get_svd_num_threads`:

Listing 6: 线程数限制函数

```

1  static int get_svd_num_threads()
2  {
3      int nthreads = SVD_NUM_THREADS;
4      if (nthreads < 1)
5      {
6          nthreads = 1;
7      }
8      if (nthreads > 8)
9      {
10         nthreads = 8;
11     }
12     return nthreads;
13 }
```

在 Pthread dynamic 和 static 实现中, 若实际使用线程数为 T , 程序只额外创建 $T-1$ 个 worker 线程, 主线程本身也作为一个 worker 参与任务执行。这样可以避免主线程只负责创建线程和等待, 而不参与计算。Pthread dynamic 版本中主线程同样调用 `pthread_block_worker`; Pthread static 版本中主线程负责 `thread_id=0` 的任务。

这种设计符合多线程实验中“主线程也参与计算”的常见优化思路, 可以减少线程资源浪费。不过, 由于 GKH block 任务数量通常很少, 主线程参与计算并不能从根本上提高并行度, 只是减少了不必要的空等。

2.10 cleanup 阶段并行化

在 GKH 每轮迭代中, 程序需要调用 `cleanup_bidiagonal`, 将上二对角结构以外且绝对值小于阈值的元素置零。该函数会扫描整个矩阵, 是规则的二维循环。与 block 级 GKH 迭代不同, cleanup 阶段没有复杂的 bulge chasing 依赖, 也不需要根据活动块动态划分任务, 因此更接近普通的数据并行问题。

原始 cleanup 的主要结构如下:

Listing 7: 串行 cleanup 逻辑

```

1  for (int i = 0; i < B.rows(); ++i)
2  {
3      for (int j = 0; j < B.cols(); ++j)
4      {
5          if (j != i && j != i + 1 && std::fabs(B.at(i, j)) <= tol)
6          {
7              B.at(i, j) = 0.0;
8          }
```

```

9     }
10  }
```

由于不同行之间的写入互不冲突，本文使用 OpenMP 对外层行循环进行并行化：

Listing 8: cleanup_bidiagonal 的 OpenMP 行级并行

```

1  #pragma omp parallel for schedule(static)
2  for (int i = 0; i < rows; ++i)
3  {
4      for (int j = 0; j < cols; ++j)
5      {
6          if (j != i && j != i + 1 && std::fabs(B.at(i, j)) <= tol)
7          {
8              B.at(i, j) = 0.0;
9          }
10     }
11 }
```

这里选择 `schedule(static)`，原因是 cleanup 每一行的工作量基本相同，且循环迭代空间规则，不需要 `dynamic` 或 `guided` 的额外调度能力。`static` 调度开销低，更适合这种规则矩阵扫描。

为避免小矩阵频繁进入 OpenMP runtime 产生额外开销，实验设置了 `SVD_CLEANUP_MIN_ELEMENTS=4096`。当矩阵元素总数小于该阈值时，cleanup 仍使用串行版本；只有较大矩阵才启用 OpenMP 并行扫描。该设计与 adaptive block threshold 思路一致：只有当任务规模足够大、并行收益可能覆盖调度开销时，才启用多线程。

2.11 并行化策略小结

综上，本文的多线程优化包含两类不同粒度的并行：

- (1) **block 级任务并行**：以 GKH 迭代中非平凡活动块为任务单位，使用 OpenMP 或 Pthread 调度。该策略保持了 GKH 算法正确性，但受限于活动块数量。
- (2) **cleanup 行级数据并行**：以矩阵行为单位并行扫描，属于规则数据并行。该策略任务数量稳定，负载均衡较好，因此更容易获得实际收益。

两类并行的特点不同。block 级并行更贴近 GKH 算法结构，但可并行任务数量随矩阵收敛过程动态变化；cleanup 并行则不依赖活动块数量，适合 OpenMP 静态调度。后续实验结果表明，最终版本的主要收益并不来自单纯增加 block 级线程数，而来自 adaptive threshold 减少无效调度开销，以及 cleanup 阶段的规则行级并行。

3 实验环境与测试规范

3.1 服务器正式测试环境

正式性能测试主要在课程服务器上完成。服务器上没有安装 Intel VTune Profiler，且服务器架构为 AArch64 / ARM。因此，服务器实验主要用于获取统一测试脚本下的正确性和运行时间数据。实验设置如下：

- 操作系统：OpenEuler 2203 SP4；

- 硬件架构: AArch64 / ARMv8;
- 编译器: g++;
- 编译优化等级: -O2;
- 固定随机种子: 20260409;
- 测试脚本: `bash test.sh 1 1 <threads> -O 02 -s 20260409;`
- 线程数: 1、2、4、8。

3.2 本地 VTune 分析环境

由于课程服务器未安装 VTune, 本文将代码克隆到本地 WSL 环境, 并交叉编译为 Windows 可执行文件, 在 Windows 11 上使用 Intel VTune Profiler 分析。该部分实验主要用于定性分析热点函数、线程等待和微架构瓶颈, 不直接作为服务器性能排名依据。

本地分析环境如下:

- 操作系统: Windows 11;
- 本地命令行环境: WSL;
- 本地机器架构: x86_64;
- 交叉编译器: x86_64-w64-mingw32-g++ 9.3;
- 分析工具: Intel VTune Profiler;
- 处理器信息: Intel microarchitecture code named Raptorlake-DT, 32 logical CPUs;
- 编译参数: `-O2 -g -fno-omit-frame-pointer -std=c++17 -fopenmp -pthread`。

需要说明的是, 本地 VTune 环境与服务器环境在 ISA、操作系统、线程库和运行时上均不完全相同。因此, VTune 结果主要用于解释程序结构性瓶颈, 而不与服务器运行时间进行逐项数值等价比较。

3.3 VTune 编译脚本

本地 VTune 使用的编译脚本如下:

Listing 9: 本地 VTune 分析编译脚本

```
1  #!/bin/bash
2  set -e
3
4  CXX=x86_64-w64-mingw32-g++
5  COMMON_SRC="main.cpp gkh.cpp bidiagonalization.cpp"
6  COMMON_FLAGS="-O2 -g -fno-omit-frame-pointer -std=c++17"
7  STATIC_FLAGS="-static -static-libgcc -static-libstdc++"
8
9  mkdir -p vtune_bins
10
```

```

11 build_one() {
12     local name=$1
13     local mode=$2
14     local threads=$3
15     local cleanup=$4
16
17     echo "Building ${name}: mode=${mode}, threads=${threads}, cleanup=${cleanup}"
18
19     $CXX $COMMON_FLAGS $COMMON_SRC \
20         -o "vtune_bins/${name}.exe" \
21         -DSVD_PARALLEL_MODE=${mode} \
22         -DSVD_NUM_THREADS=${threads} \
23         -DSVD_MIN_PARALLEL_TASKS=2 \
24         -DSVD_PARALLEL_CLEANUP=${cleanup} \
25         -DSVD_CLEANUP_MIN_ELEMENTS=4096 \
26         -fopenmp \
27         -pthread \
28         -lwinpthread \
29         $STATIC_FLAGS
30 }
31
32 build_one serial 0 1 0
33 build_one pthread_t1 4 1 0
34 build_one pthread_t2 4 2 0
35 build_one pthread_t4 4 4 0
36 build_one pthread_t8 4 8 0
37 build_one pthread_static_t8 5 8 0
38 build_one omp_static_t8 1 8 0
39 build_one omp_dynamic_t8 2 8 0
40 build_one omp_guided_t8 3 8 0
41 build_one omp_guided_cleanup_t8 3 8 1

```

4 正确性验证与服务器性能结果

4.1 正确性验证

所有正式版本均通过课程脚本中的 5 个测试用例，正确性结果均为 PASS: 5 / 5。表 3 汇总了主要版本的正确性。

表 3: 主要版本正确性验证

版本	并行模式	正确性
Range-limited serial	0	PASS: 5 / 5
OpenMP naive static	1	PASS: 5 / 5
OpenMP adaptive static	1	PASS: 5 / 5
OpenMP adaptive dynamic(1)	2	PASS: 5 / 5
OpenMP adaptive guided	3	PASS: 5 / 5
Pthread dynamic	4	PASS: 5 / 5
Pthread static	5	PASS: 5 / 5
OpenMP guided + parallel cleanup	3	PASS: 5 / 5

4.2 OpenMP naive static 与 adaptive static 对比

首先测试 naive OpenMP static 版本，即每轮 GKH 迭代无条件进入 OpenMP parallel for。结果如表 4 所示。

表 4: naive static 与 adaptive static 对比

版本	GKH time / ms	block_ms / ms	cleanup_ms / ms	avg nontrivial blocks / iter
Range-limited serial	20027.8	—	—	—
OpenMP naive static	26809.7	22783.637	3925.037	1.012
OpenMP adaptive static	22870.8	18489.663	4274.858	1.012

可以看到, naive OpenMP static 版本明显慢于串行版本。其主要原因不是计算量增加, 而是每轮 GKH 迭代都进入 OpenMP 并行区。当任务数很少, 尤其是只有一个非平凡 block 时, 线程创建、调度、同步和等待的开销无法被并行计算收益抵消。adaptive static 通过 SVD_MIN_PARALLEL_TASKS=2 避免了单任务并行, 因此相比 naive static 明显改善, 但仍慢于 range-limited serial。

4.3 OpenMP 调度策略对比

在 adaptive 框架下, 本文比较了 OpenMP static、dynamic(1)、guided 三种调度策略。结果见表 5。

表 5: OpenMP adaptive 调度策略对比

版本	GKH time / ms	block_ms / ms	cleanup_ms / ms	total_ms / ms
OpenMP adaptive static	22870.8	18489.663	4274.858	22870.283
OpenMP adaptive dynamic(1)	22323.4	18172.778	4046.101	22322.916
OpenMP adaptive guided	22303.8	18513.414	3686.257	22303.232

dynamic(1) 和 guided 均略快于 static, 说明在少数多 block 迭代中, 动态或 guided 调度可以更灵活地处理不同 block 大小带来的负载差异。然而三者差距很小, 说明调度策略不是决定性能的主因。根本原因仍然是 GKH 迭代中可同时执行的非平凡 block 数量太少。

4.4 Pthread dynamic 与 Pthread static 对比

Pthread 部分比较了 dynamic 原子任务队列和 static 固定任务划分。结果见表 6。

表 6: Pthread 调度策略对比

版本	GKH time / ms	block_ms / ms	说明
Pthread dynamic adaptive	22864.2	18744.203	原子任务队列
Pthread static adaptive	23063.5	–	固定轮转分配

Pthread dynamic 略快于 Pthread static，符合预期。dynamic 调度允许线程在完成当前 block 后继续领取任务，因此更适合任务大小不均的场景；static 调度开销更低，但在 block 大小差异较大时容易出现负载不均。不过，由于大部分迭代中只有一个非平凡 block，两者差距仍然有限。

4.5 block evolution profile 分析

大规模样例的 profiling 数据如表 7 所示。

表 7: 大规模样例活动块统计

指标	数值
GKH iterations	1425
total blocks	640527–640529
nontrivial blocks	1442
max block size	1000
avg nontrivial blocks per iteration	1.012

该结果是理解全部性能现象的关键。平均每轮迭代只有约 1.012 个非平凡 block，说明绝大多数迭代中只有一个真正需要处理的活动块。换言之，即使设置 8 个线程，在大多数迭代中也只有一个线程能执行 `one_block_step`，其余线程只能等待。因此，block 级并行的理论上限很低。

4.6 parallel cleanup 线程扩展性

由于 block 级并行受限，本文进一步对 `cleanup_bidiagonal` 进行 OpenMP 行级并行化，并测试 1、2、4、8 线程下的性能。结果见表 8。

表 8: OpenMP guided + parallel cleanup 线程扩展性

线程数	bidiag / ms	GKH / ms	cleanup_ms	block_ms	total_ms	speedup
1	3398.66	28169.9	3470.756	24597.032	28169.208	1.000
2	3795.10	23681.8	2650.825	20900.275	23681.139	1.190
4	4672.28	22201.4	1043.352	21061.011	22200.770	1.269
8	3084.93	20839.2	1098.780	19619.461	20838.633	1.352

从 1 线程到 8 线程，GKH 时间从 28169.9 ms 降低到 20839.2 ms，获得约 1.35 倍加速。cleanup 时间从 3470.756 ms 降低到约 1.1 s，说明全矩阵规则扫描能够较好地受益于 OpenMP 行级并行。4

线程和 8 线程 cleanup 时间接近, 说明该部分在若干线程后开始受到访存带宽、cache 行为或运行时调度开销影响, 无法继续线性下降。

另一方面, 8 线程下 block_ms 仍为 19619.461 ms, 占总时间绝大部分。这再次说明最终瓶颈仍然是 block-step 阶段, 而 block-step 的并行性受算法结构限制。

4.7 最终版本对比

表 9 汇总主要版本的代表性结果。

表 9: 主要版本代表性性能对比

版本	GKH time / ms	相对说明	主要结论
Range-limited serial	20027.8	基准	串行范围优化后已较强
OpenMP naive static	26809.7	变慢	并行区固定开销过大
OpenMP adaptive static	22870.8	改善	避免单任务并行
OpenMP adaptive dynamic(1)	22323.4	略优	少数多块迭代负载均衡更好
OpenMP adaptive guided	22303.8	略优	与 dynamic 接近
Pthread dynamic adaptive	22864.2	接近 OpenMP static	原子任务队列收益有限
Pthread static adaptive	23063.5	略慢	固定分配负载均衡较弱
OpenMP guided + parallel cleanup	20839.2	最优之一	cleanup 并行带来稳定收益

需要注意的是, OpenMP guided + parallel cleanup 的 8 线程版本在一次 lock compare 实验中测得 GKH time 约为 22022 ms, 而在线程扩展性实验中测得约 20839.2 ms。服务器共享环境下, 节点负载、缓存状态和调度情况会带来一定波动。因此本文更关注趋势: parallel cleanup 能显著降低 cleanup_ms, 但总时间改善受 block_ms 限制。

5 VTune 性能剖析

5.1 VTune 分析目的与环境说明

为了进一步解释服务器端性能实验中出现的现象, 本文使用 Intel VTune Profiler 对若干代表性版本进行了 Hotspots、Threading、Microarchitecture Exploration 和 Memory Access 分析 [4, 5]。需要特别说明的是, 课程服务器为 AArch64 / ARM 架构, 且服务器上未安装 VTune; 因此 VTune 分析是在本地 x86_64 Windows 环境中完成的。本地使用 x86_64-w64-mingw32-g++ 将程序交叉编译为 Windows 可执行文件, 然后通过 VTune 进行采样分析。

因此, 本节 VTune 数据的作用不是替代服务器端正式计时, 也不是与服务器端运行时间做绝对数值对应, 而是用于回答以下问题:

- (1) 程序主要热点函数是否与服务器 profiling 中的 GKH 阶段瓶颈一致;
- (2) OpenMP 和 Pthread 版本是否真正创建了多线程;
- (3) 多线程版本 CPU 利用率低的原因是线程没有创建, 还是线程创建后缺少可执行任务;
- (4) 程序底层瓶颈更接近前端取指、分支预测、后端执行, 还是内存访问;
- (5) range-limited update 和 parallel cleanup 是否改变了主要访存行为。

由于本地 VTune 环境和服务器运行环境在 ISA、操作系统、线程库和 OpenMP runtime 上均不完全相同，本文只使用 VTune 分析热点结构和瓶颈趋势，而不将 VTune 的秒级时间与服务器端 `test.sh` 输出进行直接等价比较。

5.2 Hotspots 分析

Hotspots 分析用于观察程序中 CPU Time 集中在哪些函数上。表 10 汇总了串行版本、OpenMP guided + parallel cleanup 版本以及 Pthread static 8 线程版本中的代表性热点函数。

表 10: VTune Hotspots 代表性结果

版本	热点函数	CPU Time / s	占比
serial	<code>apply_right_cols</code>	4.875	49.3%
serial	<code>apply_right_cols_range</code>	1.522	15.4%
serial	<code>Matrix::operator*</code>	0.989	10.0%
serial	<code>apply_left_rows_range</code>	0.474	4.8%
OpenMP guided + cleanup	<code>apply_right_cols</code>	4.822	46.9%
OpenMP guided + cleanup	<code>apply_right_cols_range</code>	1.476	14.4%
OpenMP guided + cleanup	<code>Matrix::operator*</code>	1.129	11.0%
OpenMP guided + cleanup	<code>WaitForMultipleObjects</code>	0.979	9.5%
OpenMP guided + cleanup	<code>apply_left_rows_range</code>	0.324	3.2%
Pthread static t8	<code>apply_right_cols</code>	4.408	45.2%
Pthread static t8	<code>apply_right_cols_range</code>	1.636	16.8%
Pthread static t8	<code>Matrix::operator*</code>	0.996	10.2%
Pthread static t8	<code>apply_left_rows_range</code>	0.478	4.9%

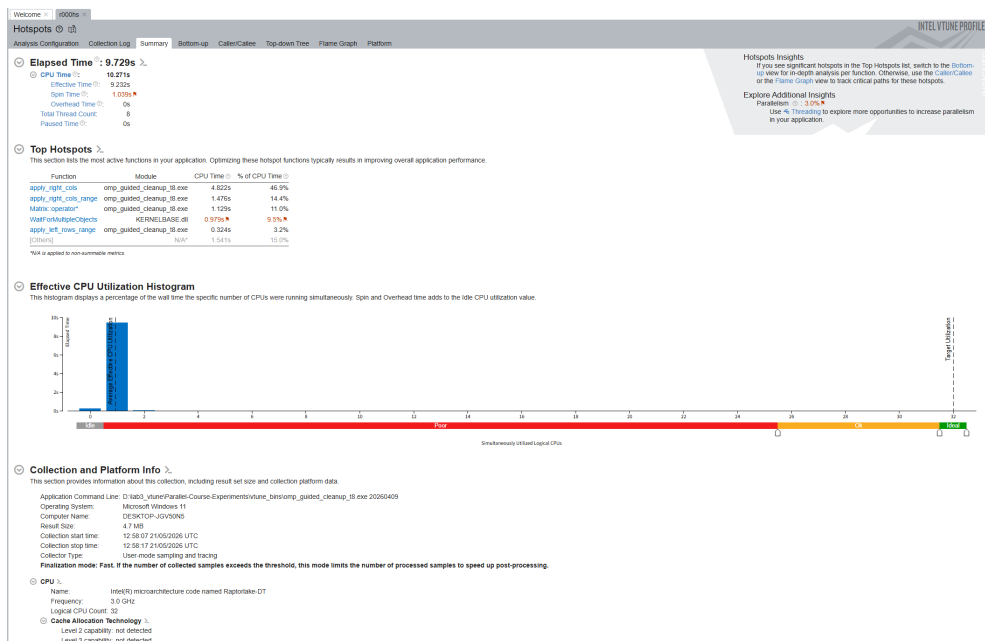


图 1: OpenMP guided + parallel cleanup 的 VTune Hotspots Summary 截图

从表 10 可以看出，无论是串行版本、OpenMP 版本还是 Pthread 版本，最大的计算热点始终是 `apply_right_cols`。该函数负责对矩阵的两列进行 Givens 旋转更新，主要用于累积 U 和 V 的正交变换。由于矩阵底层按行主序存储，按列遍历时相邻访问之间跨越一整行，空间局部性较差，因此该函数同时具有较高计算量和较重访存压力。

`apply_right_cols_range` 是第二类主要热点。该函数用于对 B 的局部 block 区间进行范围化列更新，相比原始整列更新减少了不必要的矩阵访问。它仍然占据一定 CPU Time，是因为每次 `one_block_step` 中都会频繁调用 Givens 旋转更新 B 的局部结构。

需要注意的是，OpenMP guided + cleanup 版本中出现的 `WaitForMultipleObjects` 并不是数值计算函数，而是 Windows / OpenMP runtime 层面的线程等待函数。它的出现说明该版本确实创建了多线程，并且存在 worker 线程等待同步或等待任务的情况。这一现象与后续 Threading 分析一致：多线程已经被启用，但由于可并行任务数量不足，线程无法长时间保持有效计算状态。

5.3 Threading 分析

Threading 分析用于观察线程数量、线程状态、等待时间和有效 CPU 利用率。表 11 汇总了代表性版本结果。

表 11: VTune Threading 代表性结果

版本	Elapsed Time / s	Effective CPU Utilization	Total Thread Count
serial	10.059	3.0%, 0.971 / 32 logical CPUs	1
OpenMP static t8	10.248	3.0%, 0.973 / 32 logical CPUs	8
OpenMP dynamic t8	10.552	3.0%, 0.973 / 32 logical CPUs	8
OpenMP guided t8	9.857	3.0%, 0.972 / 32 logical CPUs	8
OpenMP guided + cleanup t8	10.286	2.9%, 0.939 / 32 logical CPUs	8
Pthread t1	9.976	3.1%, 0.977 / 32 logical CPUs	1
Pthread t2	9.779	3.0%, 0.971 / 32 logical CPUs	15
Pthread t4	9.914	3.1%, 0.977 / 32 logical CPUs	19
Pthread static t8	9.908	3.0%, 0.974 / 32 logical CPUs	19

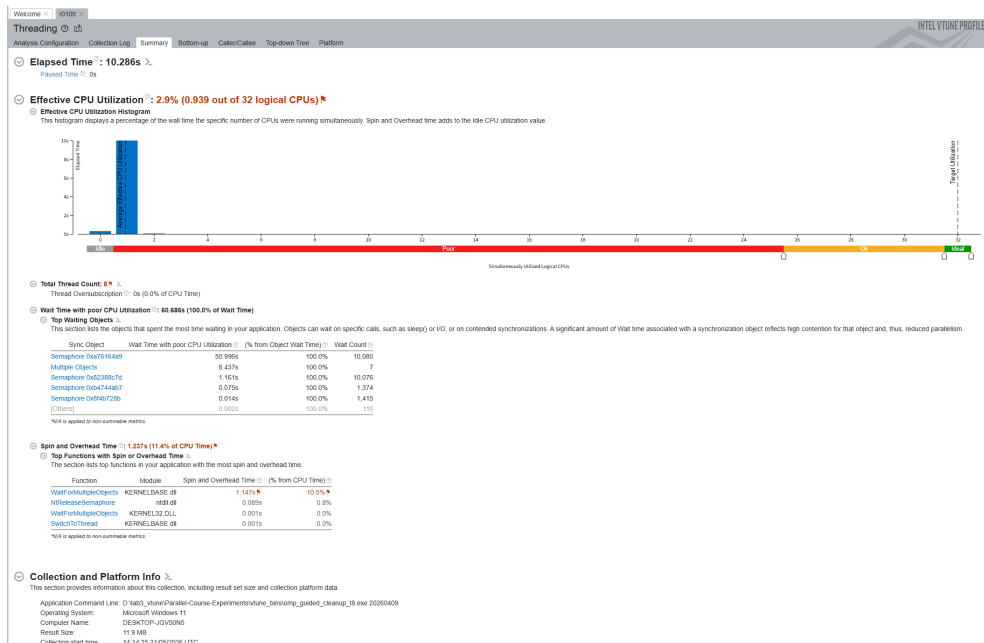


图 2: OpenMP guided + parallel cleanup 的 VTune Threading Summary 截图

表 11 中最重要的现象是：多线程版本的 Total Thread Count 明显增加，但 Effective CPU Utilization 仍然接近 1 个逻辑 CPU。这里的 3% 并不是“程序没有运行”，而是因为本地机器共有 32 个 logical CPUs，若程序平均只使用约 1 个 logical CPU，则有效利用率约为 $1/32 \approx 3.125\%$ 。因此，serial 版本的 3.0% 是合理的单线程表现；而 OpenMP 和 Pthread 版本仍接近 3.0%，说明多线程版本虽然创建了线程，但没有形成持续的多核并行执行。

这并不表示本文没有实现多线程。OpenMP static/dynamic/guided 版本的 Total Thread Count 均为 8，说明 OpenMP worker 线程已经创建；Pthread t2、t4 和 Pthread static t8 的 Total Thread Count 也明显大于 1，说明 Pthread worker 线程已经被创建。Pthread 版本的 Total Thread Count 高于设置线程数，是因为该实现采用动态线程方式，在触发并行时创建 worker 并在任务完成后 join。VTune 统计的是整个运行过程中出现过的累计线程数量，而不是某一瞬间同时活跃的线程数量。

Threading 结果真正说明的是：线程已经存在，但缺少足够的可执行任务。OpenMP 版本中，worker 线程在 timeline 上存在大量等待状态；Pthread 版本中，主线程承担了绝大部分有效计算，worker 线程只在少数多 block 迭代中短暂参与。该结果与服务器端 block profiling 完全一致：在 1000×1000 样例中，1425 次 GKH 迭代仅产生 1442 个非平凡活动块，平均每轮非平凡块数约为 1.012。也就是说，大多数轮次只有一个 block task，即使创建了 8 个线程，也只有一个线程能执行 one_block_step，其他线程只能等待。

因此，Threading 分析证明：本实验确实实现了 OpenMP 和 Pthread 多线程版本，但 GKH block 级任务数量不足，导致多线程无法长期保持高 CPU 利用率。这是算法级并行度限制，而不是线程 API 使用错误。

5.4 Microarchitecture Exploration 分析

Microarchitecture Exploration 分析用于观察程序是否受到前端取指、分支预测、后端执行资源或内存层次结构限制 [5]。表 12 汇总了代表性版本结果。

表 12: VTune Microarchitecture Exploration 代表性结果

版本	CPI Rate	Retiring	Front-End Bound	Bad Speculation	Back-End Bound	Memory Bound
serial	0.469	32.1%	2.9%	0.0%	69.9%	36.3%
Pthread t8	0.447	33.3%	2.9%	0.0%	67.8%	33.6%
OpenMP guided t8	0.442	33.8%	2.9%	3.7%	59.6%	31.3%
OpenMP guided + cleanup t8	0.448	32.7%	3.4%	3.7%	60.2%	31.6%
Pthread static t8	0.457	31.9%	3.0%	1.0%	64.1%	35.2%

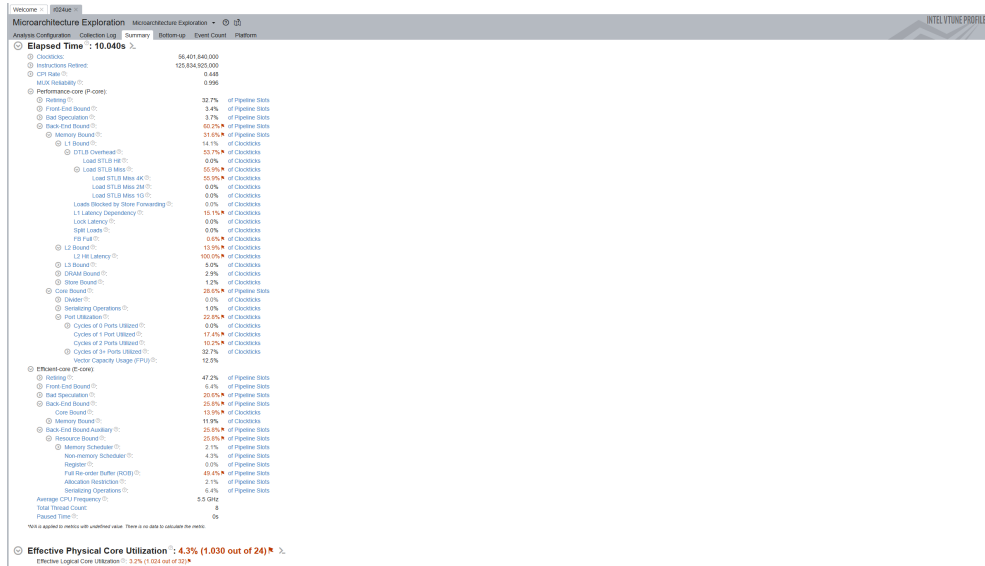


图 3: OpenMP guided + parallel cleanup 的 VTune Microarchitecture Exploration Summary 截图

结果显示, 各版本 Front-End Bound 均较低, 通常只有 3% 左右, Bad Speculation 也不高。这说明程序主要不是受取指、译码或分支预测限制。相反, 各版本 Back-End Bound 均较高, 通常在 60% 以上, 说明 CPU 后端执行资源、数据依赖和内存层次结构是主要限制因素。

Memory Bound 约为 31%–36%, 说明访存行为对性能有明显影响。结合 Hotspots 结果可知, 主要访存压力来自 `apply_right_cols` 和 `apply_right_cols_range`。其中 `apply_right_cols` 对矩阵列方向进行遍历, 而矩阵底层采用行主序存储, 因此该访问模式具有较大的 stride, 容易造成 cache locality 较差和 TLB 压力较高。

值得注意的是, OpenMP guided 和 OpenMP guided + cleanup 版本的 Back-End Bound 相比 serial 略低, 但 Effective CPU Utilization 并未显著提高。这说明 parallel cleanup 和调度策略确实改变了一部分执行行为, 但并未从根本上改变主瓶颈。主瓶颈仍然集中在 GKH block-step 内部的列更新和有限 block 并行度上。

5.5 Memory Access 分析

Memory Access 分析进一步显示程序在 DRAM 访问、LLC miss 和内存层次上的表现。表 13 汇总了代表性版本结果。

表 13: VTune Memory Access 总体指标

版本	Elapsed / s	Memory Bound	DRAM Bandwidth Bound	DRAM Transferred / GB	LLC Miss Count
serial	12.250	37.2%		112.641	53.119M
Pthread t8	11.928	37.3%	1.6%	99.198	47.870M
OpenMP static t8	11.777	37.7%	0.8%	98.028	43.468M
OpenMP guided + cleanup t8	11.073	37.4%	1.4%	91.330	45.119M

表 14: VTune Memory Access 数据传输指标

版本	DRAM Read / GB	DRAM Write / GB	Avg Read GB/s	Avg Write GB/s
serial	72.426	40.215	5.912	3.283
Pthread t8	61.748	37.450	5.177	3.140
OpenMP static t8	60.956	37.072	5.176	3.148
OpenMP guided + cleanup t8	56.862	34.468	5.135	3.113

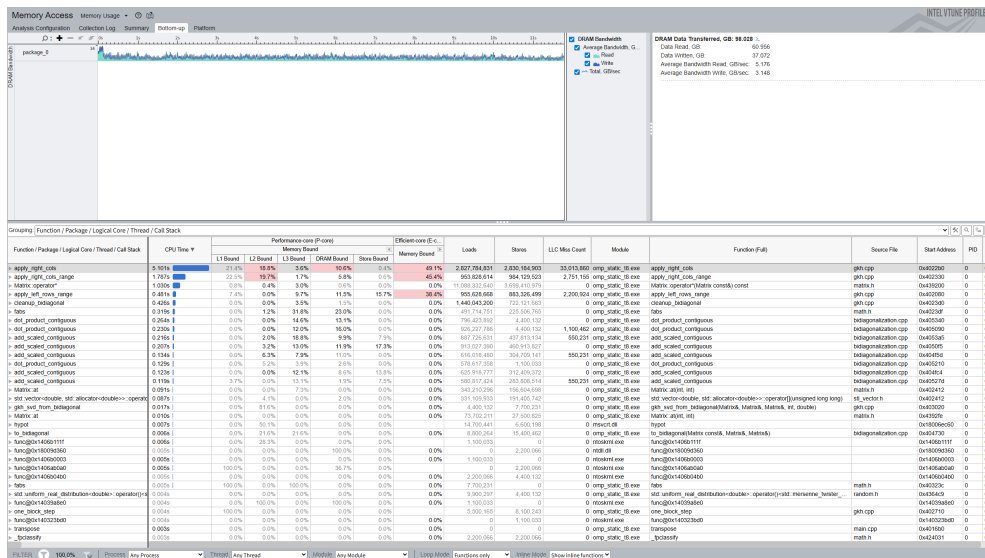


图 4: OpenMP guided + parallel cleanup 的 VTune Memory Access Bottom-up 截图

表 13 显示，各版本 Memory Bound 均在 37% 左右，说明并行调度并没有改变程序访存受限的基本特征。同时，各版本 DRAM Bandwidth Bound 都很低，最高也只有 1.6%。这说明程序并不是典型的 DRAM 带宽打满，而更可能是受到 cache 层次、TLB、访问步长和数据依赖影响。

从 DRAM 数据传输量看，OpenMP guided + cleanup 版本的 DRAM Transferred 从 serial 的 112.641 GB 降低到 91.330 GB，说明 range-limited B update 和 parallel cleanup 对减少部分无效访存是有效的。但其 Memory Bound 仍为 37.4%，没有明显低于 serial。这表明虽然总访存量有所下降，核心列访问模式仍然没有根本改变。

Memory Access 的 Bottom-up 结果进一步显示，`apply_right_cols` 是最主要的访存热点。例如在 OpenMP guided + cleanup 版本中，`apply_right_cols` 的 CPU Time 约为 5.161 s，Loads 约为 2.870B，Stores 约为 2.892B，LLC Miss Count 约为 38.516M；`apply_right_cols_range` 的 CPU Time 约为 1.651 s，LLC Miss Count 约为 2.201M。这说明 range-limited B update 的局部性优于全局列更新，但 U/V 的累积更新仍必须执行整列旋转，因此 `apply_right_cols` 仍是最大访存瓶颈。

5.6 VTune 结果与服务器实验的对应关系

虽然 VTune 在本地 x86 平台完成，而服务器正式测试在 AArch64 平台完成，但两者结论在趋势上是一致的。服务器端 profiling 显示，GKH 阶段中 `one_block_step` 占据主要时间；VTune Hotspots 则进一步指出 `one_block_step` 内部的 `apply_right_cols` 和 `apply_right_cols_range` 是主要热点。服务器端 block evolution 显示平均每轮非平凡 block 数约为 1.012；VTune Threading 则显示多线程版本虽然创建了 worker 线程，但平均有效 CPU 利用率仍接近单线程。服务器端线程扩展实验显示 parallel cleanup 能降低 `cleanup_ms`；VTune Memory Access 也显示 OpenMP guided + cleanup 版本的 DRAM Transferred 低于 serial 和 Pthread t8。

因此，VTune 分析不是孤立的本地实验，而是对服务器端性能结果的解释补充。二者共同说明：本文实现了多线程并行框架，但当前 GKH 算法在该测试样例下实际可并行任务不足，同时主要热点函数存在较强访存瓶颈，因此难以获得线性加速。

5.7 VTune 结论

综合 Hotspots、Threading、Microarchitecture 和 Memory Access，可以得到以下结论：

- (1) 本文确实实现了 OpenMP 和 Pthread 多线程版本。VTune 中 OpenMP 版本 Total Thread Count 为 8，Pthread 版本也出现多个 worker 线程，证明并行框架已经启用。
- (2) 多线程版本平均 CPU 利用率仍接近 1 个 logical CPU，原因不是线程没有创建，而是大多数 GKH 迭代只有一个非平凡 block，无法为多个线程提供持续任务。
- (3) 主要计算热点集中在 Givens 旋转相关函数，尤其是 `apply_right_cols`。该函数负责 U/V 的全局列更新，是各版本中最稳定的热点。
- (4) OpenMP 和 Pthread 的线程等待现象明显，说明当前性能瓶颈主要来自算法级并行度不足，而不是某一种线程 API 的具体实现问题。
- (5) 微架构上程序主要表现为 Back-End Bound 和 Memory Bound，前端取指和分支预测并不是主要瓶颈。
- (6) Memory Access 结果说明程序不是典型 DRAM 带宽饱和，而是受到矩阵列访问、cache 层级、TLB 和访问局部性共同影响。
- (7) parallel cleanup 能降低规则矩阵扫描部分的开销，并减少部分 DRAM 数据传输量，但无法消除 GKH block-step 中 `apply_right_cols` 的主瓶颈。

6 总结与思考

本实验围绕矩阵 SVD 分解中的 Golub-Kahan SVD iteration 展开 Pthread 与 OpenMP 多线程优化。与普通矩阵乘法、向量加法等规则数据并行问题不同，GKH 迭代属于典型的数值线性代数迭代算法，其并行性并不直接表现为某个大循环可以简单拆分给多个线程。本实验最初的直觉是：GKH 迭代中存在多个由 `split_active_blocks` 生成的活动块，因此可以通过 OpenMP 或 Pthread 对不同 block 并行处理，从而获得明显加速。然而完整实验结果表明，数值算法中的“理论可并行”并不等价于“实际可获得高性能加速”。虽然活动块之间在数学上可以并行，但在当前测试样例中，大多数迭代只有一个非平凡活动块，导致 block 级并行缺少足够任务供多个线程同时执行。

6.1 对算法并行性的认识

通过本实验，我对 GKH 迭代的并行性有了更深入的理解。GKH 的核心过程是 bulge chasing：从活动块左端引入 bulge，再通过交替的右 Givens 旋转和左 Givens 旋转将其逐步追赶到块右端。该过程存在明显的前后依赖，后续旋转参数需要依赖前一次旋转更新后的矩阵元素。因此，单个活动块内部并不适合直接使用 `parallel for` 并行化。本文最终选择 block 间并行，而不是 block 内并行，是在数值正确性和并行收益之间做出的折中。

从 block profiling 结果看， 1000×1000 大规模样例中共有 1425 轮 GKH 迭代，但总共只有 1442 个非平凡活动块，平均每轮非平凡活动块数约为 1.012。这意味着绝大多数迭代中只有一个真正需要处理的 block。该结果解释了后续几乎所有性能现象：OpenMP static、dynamic、guided 之间差异较小，Pthread dynamic 和 static 差异也不明显，多线程版本 CPU 利用率仍接近单线程。这说明多线程优化首先受限于算法本身能否提供足够并行任务，而不仅仅取决于是否使用了某个线程库。

6.2 对 naive 并行的反思

朴素 OpenMP static 版本是本实验中非常重要的负面对照。它直接对每轮活动块任务列表使用 `#pragma omp parallel for schedule(static)`。从程序形式上看，这已经是一个正确的 OpenMP 并行版本；但实验结果表明，它不仅没有加速，反而比 range-limited 串行版本更慢。这说明如果不分析任务粒度、任务数量和调度开销，直接套用 OpenMP 指令可能会得到相反的效果。

这一点对并行程序设计很重要。并行化并不是把循环前面加上 `parallel for` 就结束了，而是要判断每次进入并行区时是否有足够任务、每个任务是否足够重、线程之间是否需要同步、调度开销是否会抵消计算收益。对于 GKH 迭代而言，大部分轮次只有一个 block task，进入 OpenMP 并行区只会让多个线程参与调度和同步，却无法增加实际计算线程数。因此，naive OpenMP static 版本变慢并不是实现错误，而是并行粒度和算法特征不匹配。

adaptive threshold 的引入正是针对这一问题。通过设置 `SVD_MIN_PARALLEL_TASKS=2`，程序只有在当前轮次至少有两个非平凡 block 时才启用并行，否则直接串行执行。这一优化虽然很简单，但它体现了一个重要原则：并行运行时有固定成本，只有当任务规模足够大时才值得启用多线程。实验中 adaptive static 明显快于 naive static，说明减少无效并行区开销比盲目增加线程更加有效。

6.3 对 OpenMP 与 Pthread 调度策略的认识

本实验分别实现并比较了 OpenMP static、OpenMP dynamic(1)、OpenMP guided(1)、Pthread dynamic 原子任务队列和 Pthread static 固定任务分配。不同调度策略的设计目标不同：static 调度开销低，但负载均衡能力较弱；dynamic 调度负载均衡能力更强，但任务领取开销更高；guided 调度试图在二者之间折中；Pthread dynamic 通过原子任务编号模拟动态任务队列；Pthread static 则通过固定轮转方式分配任务。

实验结果显示，OpenMP dynamic 和 guided 相比 static 略有优势，Pthread dynamic 也略优于 Pthread static，但这些差距都不大。这说明在当前 GKH block 任务分布下，调度策略不是最主要矛盾。原因在于，调度策略只能解决“已有多个任务时如何分配”的问题，不能解决“大多数迭代根本没有多个任务”的问题。当任务列表长度为 1 时，无论使用 static、dynamic、guided 还是手写 Pthread 动态队列，都只能有一个线程执行有效计算。

因此，本实验让我认识到：负载均衡问题必须和任务数量问题区分开。dynamic 或 guided 调度确实能改善少数多 block 迭代中的负载不均，但如果算法大部分时间只产生一个 block，则调度

策略的改进空间天然有限。这个结论也解释了为什么 OpenMP 和 Pthread 在本实验中的性能表现接近：两者都被同一个算法级瓶颈限制。

6.4 对 cleanup 并行化的认识

与 block-step 不同，`cleanup_bidiagonal` 是规则的全矩阵扫描，行之间基本独立，任务粒度稳定。因此，它比 GKH active block 更适合使用 OpenMP 行级并行。实验结果表明，parallel cleanup 可以显著降低 cleanup 阶段耗时，在线程扩展实验中，cleanup 时间从 1 线程的约 3470.756 ms 降低到 8 线程的约 1098.780 ms。最终 8 线程 OpenMP guided + parallel cleanup 版本相对 1 线程版本取得约 1.35 倍加速。

不过，cleanup 并行化并没有让整个程序获得线性加速。原因在于，cleanup 虽然可并行性较好，但它不是总时间中唯一的主要部分。8 线程时，`block_ms` 仍约为 19619.461 ms，远高于 `cleanup_ms`。也就是说，parallel cleanup 优化的是一个规则、可并行、但占比有限的阶段；真正占主导的 block-step 阶段仍然受活动块数量和内部顺序依赖限制。

这一结果说明，并行优化需要同时考虑“某段代码是否容易并行”和“该段代码在总时间中占比多大”。cleanup 是一个适合并行的局部优化点，但它无法完全弥补 GKH 主体阶段并行性不足的问题。

6.5 对 VTune 性能分析的认识

本实验不仅比较了服务器端运行时间，还使用 Intel VTune Profiler 对本地 x86 版本进行了 Hotspots、Threading、Microarchitecture Exploration 和 Memory Access 分析。虽然本地 VTune 环境与服务器 AArch64 环境不同，不能将时间数值直接等价比较，但 VTune 对理解程序瓶颈非常有帮助。

Hotspots 分析显示，各版本主要热点始终集中在 `apply_right_cols`、`apply_right_cols_range` 和 `Matrix::operator*`。其中 `apply_right_cols` 长期占据最高 CPU Time，说明 Givens 旋转中对 U/V 的列更新是程序最稳定的热点。Threading 分析显示，多线程版本确实创建了 worker 线程，但平均 Effective CPU Utilization 仍接近一个 logical CPU。这进一步证明，性能不足不是因为没实现多线程，而是线程创建后缺少足够任务可执行。

Microarchitecture Exploration 显示程序主要表现为 Back-End Bound 和 Memory Bound，而不是 Front-End Bound 或 Bad Speculation。这说明瓶颈主要来自后端执行资源、数据依赖和内存层次结构。Memory Access 分析进一步显示，各版本 Memory Bound 基本都在 37% 左右，DRAM Bandwidth Bound 却很低，说明程序并不是简单的内存带宽打满，而是受到列方向访问、cache 层级、TLB 和访问局部性的共同影响。

这些分析帮助我避免了一个常见误区：只看“是否开了 8 个线程”是不够的。即使程序创建了 8 个线程，如果线程大部分时间在等待，那么有效并行度仍然很低。同样，只看总时间也不够，还需要知道时间花在哪些函数上、线程为什么等待、CPU 是前端受限还是后端受限、访存是不是带宽瓶颈。VTune 的多角度分析让本实验的结论更加可信。

6.6 局限性与后续优化方向

本实验仍然存在一些局限。首先，正式性能测试在 AArch64 服务器上进行，而 VTune 分析在本地 x86 平台完成。两套环境的硬件架构、操作系统和线程运行时均不同，因此 VTune 结果主要用于解释趋势，而不能完全替代服务器端 profiling。其次，当前 Pthread 版本采用的是在需要并行

时创建 worker 并 join 的动态线程方式，而不是完整的长生命周期线程池。由于当前任务数量本身很少，线程池未必能带来质变，但如果进一步研究更细粒度或更多轮次任务，静态线程池可能值得尝试。

从算法角度看，后续优化可以考虑以下方向：

- (1) **优化矩阵存储布局**。当前矩阵采用行主序存储，而 `apply_right_cols` 需要按列访问，访存局部性较差。可以考虑为 U 、 V 使用转置存储或列友好的布局。
- (2) **批量化 Givens 更新**。当前每次旋转都会立即更新 U 或 V 的两列，导致反复整列扫描。后续可以考虑延迟累积若干 Givens 旋转，再批量应用到 U/V ，减少内存访问次数。
- (3) **引入 blocked 或 tiled 更新**。将矩阵更新组织为更 cache-friendly 的块操作，有可能改善 cache locality，并为 SIMD 或多线程提供更规则的计算结构。
- (4) **研究更高层次的并行算法**。当前只利用了 active block 间并行，而没有改变 GKH 算法本身。如果希望获得更高并行度，可能需要研究分治 SVD、Jacobi SVD 或 blocked bidiagonal SVD 等更适合并行的算法策略。
- (5) **构建长生命周期线程池**。对 Pthread 版本而言，可进一步尝试静态线程池配合 barrier/condition variable，减少频繁创建与销毁线程的开销。

总体来看，本实验的价值并不只在于得到某个最高加速比，而在于通过多轮实现和 profiling 分析，逐步定位了 GKH 迭代并行化的边界：当前实现中既存在算法级并行度不足，也存在列访问导致的访存瓶颈。因此，后续若要继续提升性能，需要从算法结构和数据布局两方面入手，而不能只依赖更换线程 API 或增加线程数。

7 结论

本文完成了矩阵 SVD 分解程序中 GKH 迭代阶段的 Pthread 与 OpenMP 多线程优化实验，并对不同并行策略进行了系统比较。实验从串行 GKH 迭代出发，逐步实现了 range-limited block 更新、OpenMP static/dynamic/guided 调度、Pthread dynamic/static 调度、自适应并行阈值和 cleanup 行级并行化。所有主要版本均通过课程脚本中的 5 个测试样例，说明多线程优化没有破坏 SVD 分解的正确性。

本文的主要结论如下：

- (1) **GKH 迭代存在 block 级并行机会，但并行度有限**。不同活动块由已经收敛为零的超对角线元素分隔，可以作为独立任务并行处理；但单个活动块内部的 bulge chasing 具有强顺序依赖，不能简单并行化。在 1000×1000 样例中，平均每轮非平凡活动块数约为 1.012，说明实际 block 级并行度很低。
- (2) **范围化矩阵更新是 block 并行正确性的关键**。原始行/列旋转函数可能对整行或整列进行更新，在 block 并行时可能写入其他 block 区域。本文引入 `apply_right_cols_range` 和 `apply_left_rows_range`，将 B 的更新限制在当前活动块范围内，从而避免不同 block 之间的数据竞争。 U 和 V 的更新则依赖不同 block 对应列范围互不重叠来保证安全。
- (3) **naive OpenMP static 并行会因任务数量不足而变慢**。实验表明，直接在每轮 GKH 迭代中使用 `parallel for` 会带来明显调度和同步开销。当当前轮只有一个非平凡 block 时，多个线程无法同时工作，OpenMP runtime 开销反而使程序变慢。

- (4) **adaptive threshold** 能减少无效并行开销。通过设置 `SVD_MIN_PARALLEL_TASKS=2`，程序在任务数不足时自动退化为串行执行，从而避免单任务并行区的固定开销。该优化显著改善了 naive OpenMP static 版本的性能。
- (5) **OpenMP dynamic/guided 和 Pthread dynamic 的差异有限**。dynamic 和 guided 调度可以在少数多 block 迭代中改善负载均衡，Pthread dynamic 也比 Pthread static 略好；但由于多 block 迭代数量太少，不同调度策略之间的性能差距不大。调度策略不是主要瓶颈，算法级任务数量不足才是根本限制。
- (6) **cleanup 行级并行是本实验中较稳定的加速来源**。`cleanup_bidiagonal` 是规则全矩阵扫描，行之间基本独立，适合 OpenMP static 行级并行。线程扩展实验中，8 线程 OpenMP guided + parallel cleanup 版本相对 1 线程版本获得约 1.35 倍加速，主要收益来自 cleanup 阶段和少量 block 并行收益。
- (7) **VTune 证明多线程已实现，但有效并行度受限**。Threading 分析显示 OpenMP 和 Pthread 版本均创建了多个线程，但平均有效 CPU 利用率仍接近单线程。这说明问题不是线程没有创建，而是大多数迭代中任务数量不足，导致 worker 线程处于等待状态。
- (8) **主要热点是 Givens 旋转中的列更新**。Hotspots 和 Memory Access 分析显示，`apply_right_cols` 是各版本中最稳定的主要热点。由于矩阵采用行主序存储，列方向访问具有较差空间局部性，使程序表现出明显的后端和访存瓶颈。
- (9) **当前程序不是单纯 DRAM 带宽瓶颈**。Memory Access 结果显示各版本 Memory Bound 约为 37%，但 DRAM Bandwidth Bound 较低。这说明瓶颈主要来自访问局部性、cache/TLB 层级和数据依赖，而不是内存带宽被完全打满。

综上，本文实现了符合实验要求的 Pthread 与 OpenMP 多线程版本，并对调度策略、线程数扩展、同步开销、热点函数和访存瓶颈进行了系统分析。实验表明，对于 GKH 这类具有复杂依赖关系的数值迭代算法，多线程优化不能只关注线程 API 的使用，而必须结合算法结构、任务粒度、数据布局和硬件特征共同设计。当前实现已经证明 block 级并行和 cleanup 行级并行的可行性，同时也揭示了进一步优化的方向：若希望获得更高加速比，需要从矩阵存储布局、批量 Givens 更新、blocked/tiled 数据结构或更适合并行的 SVD 算法策略入手。

总体而言，本实验说明：对于数值线性代数程序，多线程优化不能只关注线程 API 的使用，还必须结合算法依赖、任务粒度、内存访问模式和运行时调度开销进行综合分析。当前实现已经完成了 OpenMP 与 Pthread 的多种调度方式对比，并通过 profiling 找到了主要瓶颈。后续若继续优化，可考虑改变矩阵存储布局、优化列访问、对 U/V 更新采用更 cache-friendly 的 blocked layout，或在更高层次重新设计 GKH 迭代的并行算法。

A Git 项目链接

GitHub 项目仓库: <https://github.com/ZR-1N/Parallel-Course-Experiments>

实验分支: `svd-lab3-pthread-openmp`

B 实验结果文件组织

实验开始时，为便于保存不同阶段的结果，我预先创建了如下目录结构：

Listing 10: results_lab3 初始规划目录

```

1 results_lab3/
2   00_baseline/
3   01_simd_serial/
4   02_openmp/
5   03_pthread/
6   04_lock_compare/
7   05_schedule/
8   06_thread_scaling/
9   07_block_profile/
10  08_auto_vectorization/
11  09_final/

```

需要说明的是，该目录结构是实验初期的规划，并非所有目录在后续实验中都被严格使用。随着实验推进，实际记录主要集中在串行基准、OpenMP 调度、Pthread 调度、线程扩展性、block evolution profiling 以及最终版本分析等部分。本文主要引用的结果包括：

- 01_simd_serial/: 记录从前一次 SIMD 实验继承而来的串行基准结果；
- 02_openmp/: 记录 OpenMP naive static、adaptive static、dynamic、guided 等调度策略实验；
- 03_pthread/: 记录 Pthread dynamic block-level scheduling 实验；
- 04_lock_compare/: 记录 OpenMP guided + parallel cleanup 等对照实验；
- 05_schedule/: 记录 Pthread static 与 dynamic 调度方式对比；
- 06_thread_scaling/: 记录 1、2、4、8 线程扩展性实验；
- 07_block_profile/: 记录 GKH 活动块演化 CSV 及非平凡 block 数量统计；
- 09_final/: 用于整理最终版本及报告中使用的关键结果。

其中 00_baseline/ 和 08_auto_vectorization/ 属于初始规划中的保留目录，后续并非本文主要分析对象。本文所有性能结论均以实际记录的 README、test.o、test.e、block profile CSV 和 VTune 截图为依据。

C AI 辅助编程对话记录

本实验过程中使用 AI 主要用于辅助理解 GKH 迭代的并行结构、比较 OpenMP 与 Pthread 的实现方式、分析实验结果以及整理 VTune profiling 结论。AI 没有直接替代实验测试，所有代码修改均通过课程脚本或本地 VTune 环境验证。以下记录选取了实验过程中较有代表性的若干问题。

C.1 GKH 迭代是否可以整体并行化

问题： GKH 迭代中存在多层循环，是否可以直接对 one_block_step 内部循环使用 OpenMP 或 Pthread 并行？

回答摘要： 不建议直接并行化 one_block_step 内部的 bulge chasing 循环。单个活动块内部的右 Givens 旋转和左 Givens 旋转具有前后依赖，后一位置的旋转参数依赖前一位置更新后的矩阵

元素。若强行并行化 block 内部循环，可能破坏 bulge 传播顺序并导致错误的 SVD 结果。更合理的策略是先通过 `split_active_blocks` 找到互相独立的活动块，再在不同 block 之间进行任务级并行。

C.2 为什么需要 range-limited B update

问题：如果不同活动块之间已经相互独立，为什么还需要写 `apply_right_cols_range` 和 `apply_left_rows_range`？

回答摘要：数学上的 block 独立并不自动保证代码实现中不会写出当前 block 范围。原始 `apply_right_cols` 和 `apply_left_rows` 会对整列或整行执行旋转，如果多个 block 并行执行，可能改写其他 block 所在区域，从而引入数据竞争或破坏分块结构。对 B 使用 range-limited 更新，可以保证每个 block 只写入自己的局部子矩阵范围。这样不同 block 之间不需要全局锁，也能保持矩阵更新的正确性。

C.3 OpenMP naive static 版本为什么反而变慢

问题：将 block 任务列表直接改为 `#pragma omp parallel for schedule(static)` 后，为什么运行时间比串行版本更长？

回答摘要：OpenMP 并行区存在固定的线程调度和同步开销。如果每轮 GKH 迭代中只有一个非平凡活动块，那么虽然程序进入了并行区，但实际上只有一个线程能执行有效计算，其他线程只能等待。此时并行开销大于收益。应先统计每轮任务数量，并引入最小任务数阈值。若 `task_count < SVD_MIN_PARALLEL_TASKS`，则直接串行执行，避免无效并行区开销。

C.4 OpenMP static、dynamic 和 guided 应该如何比较

问题：OpenMP 的 static、dynamic 和 guided 调度都能用于 block 任务列表，它们分别适合什么情况？

回答摘要：static 调度开销最低，适合每个任务耗时相近的循环；dynamic,1 让线程动态领取任务，适合 block 大小不均的情况，但调度开销更高；guided,1 在调度开销和负载均衡之间折中。对于 GKH block 任务，如果某些迭代中存在多个大小不同的活动块，dynamic 或 guided 可能略优。但如果大多数迭代只有一个非平凡 block，那么三种调度策略的差距都会很小，因为调度策略不能增加任务数量。

C.5 Pthread dynamic 是否比 static 更合适

问题：Pthread 版本应该采用固定任务划分，还是用原子变量实现动态任务队列？

回答摘要：两种方式都值得实现并作为对照。Pthread static 固定分配任务，开销较低，但当 block 大小不均时可能出现负载不均；Pthread dynamic 使用 `std::atomic<int>` 作为任务编号，线程完成当前任务后可以继续领取新任务，负载均衡更好，但存在原子操作和线程创建开销。对于本实验的 GKH block 任务，dynamic 理论上更灵活，但由于多数迭代任务数量很少，dynamic 相比 static 只能带来有限改善。

C.6 主线程是否应该参与 Pthread 计算

问题：Pthread 版本中，主线程创建 worker 后是否只需要等待，还是也应该参与计算？

回答摘要：建议主线程也作为一个 worker 参与任务计算。如果设置总线程数为 8，则可以额外创建 7 个 worker，主线程负责第 8 个 worker 的任务。这样可以避免主线程只负责创建和等待而浪费一个核心。对于 dynamic 版本，主线程也调用同一个 worker 函数领取任务；对于 static 版本，主线程可以负责 `thread_id=0` 的任务。

C.7 cleanup 阶段为什么适合 OpenMP 行级并行

问题：既然 block 级并行效果有限，是否还有其他部分更适合多线程优化？

回答摘要：`cleanup_bidiagonal` 是一个规则的矩阵扫描过程，主要判断非上二对角结构元素是否足够小并置零。不同行之间基本没有依赖，因此可以对外层行循环使用 OpenMP static 调度。与 GKH block-step 不同，cleanup 的任务粒度稳定、负载均衡较好，更适合数据并行。不过，小矩阵进入 OpenMP 并行区可能得不偿失，因此应设置规模阈值，例如元素数量超过 `SVD_CLEANUP_MIN_ELEMENTS` 时才启用并行 cleanup。

C.8 VTune 在服务器上无法使用时如何处理

问题：课程服务器是 AArch64 架构且没有安装 VTune，但实验要求需要使用 VTune 分析性能，应该怎么办？

回答摘要：服务器端仍然应作为正式正确性和性能计时环境，使用课程提供的 `test.sh` 输出作为主要性能数据。VTune 可以在本地 x86 平台辅助分析热点和线程行为。由于本地使用 Windows + WSL，需要将代码克隆到本地，并使用 `x86_64-w64-mingw32-g++` 编译 Windows 可执行文件，保留 `-O2 -g -fno-omit-frame-pointer` 等适合 profiling 的参数。报告中需要明确说明：VTune 结果用于解释热点、线程等待和访存趋势，不能与服务器 AArch64 运行时间做绝对数值等价比较。

C.9 本地 VTune 结果和服务器结果不一致时如何解释

问题：本地 VTune 的 elapsed time 和服务器上 `test.sh` 的时间不同，是否会影响报告结论？

回答摘要：不会，只要明确两类数据用途不同。服务器数据用于比较正式版本的正确性、总时间、线程数扩展和调度策略；本地 VTune 数据用于分析程序结构性瓶颈，例如热点函数是否集中在 `apply_right_cols`、线程是否大量等待、程序是否 Memory Bound。由于硬件架构和运行时不同，不能直接比较两边的秒数，但如果两者都指向同一结论，例如 block-step 占主导、线程等待明显、列访问是热点，则说明分析具有一致性。

C.10 如何解释 VTune 中多线程版本 CPU 利用率仍然很低

问题：OpenMP 和 Pthread 版本确实创建了多个线程，但 VTune Threading 中 Effective CPU Utilization 仍接近单线程，这是否说明多线程没有实现成功？

回答摘要：不能这样理解。VTune 中 Total Thread Count 已经显示 OpenMP 或 Pthread worker 被创建，说明多线程框架确实启用。Effective CPU Utilization 低说明这些线程没有持续执行有效计算，主要原因是可并行任务不足。结合 block profiling 结果， 1000×1000 样例中平均每轮非平凡 block 数约为 1.012，大多数迭代只有一个任务，因此其他线程只能等待。这说明瓶颈是算法级并行度不足，而不是线程 API 使用失败。

C.11 如何理解 `apply_right_cols` 长期占据热点

问题：为什么 Hotspots 和 Memory Access 中 `apply_right_cols` 总是最大热点？

回答摘要：`apply_right_cols` 用于对矩阵两列进行 Givens 旋转更新，尤其是累积 U 和 V 的正交变换。由于矩阵采用行主序存储，按列访问时相邻访问之间跨越一整行，空间局部性较差，容易造成 cache/TLB 压力。range-limited update 能减少 B 的局部更新开销，但 U/V 的全局列更新仍然需要调用 `apply_right_cols`，因此它长期是主要热点。

C.12 后续如果继续优化应该从哪里入手

问题：如果继续优化该 SVD 程序，除了继续改 OpenMP/Pthread 调度，还可以考虑哪些方向？

回答摘要：继续更换调度策略的收益可能有限，因为主要问题是 block 数量不足和列访问局部性差。更有潜力的方向包括：为 U/V 设计列友好的存储布局或转置存储；将多个 Givens 旋转延迟累积后批量应用，减少反复整列扫描；使用 blocked/tiled 数据结构改善 cache locality；研究更高层次的并行 SVD 算法，例如 blocked bidiagonalization、Jacobi SVD 或分治 SVD。这些方向比单纯增加线程数更可能带来进一步加速。

D 参考文献

参考文献

- [1] OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.1*. OpenMP Architecture Review Board, 2020.
- [2] David R. Butenhof. *Programming with POSIX Threads*. Addison-Wesley, 1997.
- [3] IEEE and The Open Group. *POSIX Threads Programming*. The Open Group. POSIX Threads API specification and related documentation.
- [4] Intel Corporation. *Intel VTune Profiler User Guide*. Intel Corporation. Performance profiling tool documentation.
- [5] Intel Corporation. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. Intel Corporation. Microarchitecture optimization reference manual.